

RETO QA AUTOMATION

API REST con Screenplay Pattern

Propuesta de Evaluación Técnica

Equipo SQA Tyba

Abril 2026

Tyba

1. Objetivo

Cada miembro del equipo debe crear un repositorio en GitHub que automatice las operaciones CRUD (POST, GET, PUT, DELETE) de una API REST pública, aplicando el patrón Screenplay con Serenity BDD, Java y Gradle.

Este reto evalúa la capacidad del equipo para estructurar correctamente un framework de automatización siguiendo los patrones y convenciones del proyecto tyba-auto-backend.

2. API Sugerida

Usar una API pública gratuita que soporte CRUD completo. Opciones recomendadas:

API	URL	Nivel
Reqres.in	https://reqres.in	Básico - Ideal para empezar
JSONPlaceholder	https://jsonplaceholder.typicode.com	Básico - Posts, comments, users
GoRest	https://gorest.co.in	Intermedio - Requiere token

3. Estructura Esperada del Repositorio

El repositorio debe seguir la siguiente organización de paquetes, alineada con el patrón Screenplay:

```

qa-rest-screenplay-challenge/
├── build.gradle
├── gradle/wrapper/
├── gradlew / gradlew.bat
├── settings.gradle
├── README.md
├── src/
│   ├── main/java/com/challenge/
│   │   ├── interactions/api/           # Get, Post, Put, Delete
│   │   ├── tasks/                     # CreateUser, GetUser, ...
│   │   ├── questions/                 # TheStatusCode, TheResponseBody
│   │   ├── models/
│   │   │   ├── request/               # POJOs con @Builder y @Data
│   │   │   └── response/              # POJOs con @JsonProperty
│   │   ├── utils/
│   │   │   ├── constants/             # Resources (enum endpoints)
│   │   │   └── config/                # EnvironmentConfig
│   └── test/
│       ├── java/com/challenge/
│       │   ├── runners/               # Suite JUnit 5
│       │   └── resources/
│       │       └── serenity.conf       # Config de ambientes

```

4. Dependencias Obligatorias

El archivo build.gradle debe incluir las siguientes dependencias:

Dependencia	Propósito
net.serenity-bdd:serenity-core	Core del framework Serenity BDD
net.serenity-bdd:serenity-screenplay	Implementación del patrón Screenplay
net.serenity-bdd:serenity-screenplay-rest	Integración REST con Screenplay
net.serenity-bdd:serenity-junit5	Runner JUnit 5 para Serenity
org.projectlombok:lombok	Reducción de boilerplate (@Data, @Builder)
tools.jackson.core:jackson-databind	Serialización/deserialización JSON
org.assertj:assertj-core	Assertions fluidas y expresivas

5. Criterios de Evaluación

La evaluación se divide en 4 categorías con un total de 100 puntos:

5.1 Arquitectura Screenplay (30 puntos)

Criterio	Pts	Descripción
Interactions	8	Clases Post, Get, Put, Delete que extiendan RestInteraction con métodos factory
Tasks	8	Al menos 4 tasks: CreateUser, GetUser, UpdateUser, DeleteUser
Questions	8	Al menos TheStatusCode y TheResponseBody como Questions reutilizables
Actor Setup	6	Actor configurado con CallAnApi.at(baseUrl) usando EnvironmentConfig

Ejemplo esperado de un Task:

```
public class CreateUser implements Task {
    private final UserRequest payload;

    @Override
    public <T extends Actor> void performAs(T actor) {
        actor.attemptsTo(
            Post.to(Resources.USERS.getValue())
                .with(request -> request
                    .contentType(ContentType.JSON)
                    .body(payload))
        );
    }

    public static CreateUser with(UserRequest payload) {
        return instrumented(CreateUser.class, payload);
    }
}
```

5.2 Cobertura de Operaciones REST (30 puntos)

Operación	Pts	Validación mínima
POST	8	Crear recurso, validar status 201, verificar campos en response body
GET	7	Consultar recurso, validar status 200, verificar estructura de respuesta
PUT	8	Actualizar recurso, validar status 200, verificar datos actualizados
DELETE	7	Eliminar recurso, validar status 204 o 200

Ejemplo esperado de un test:

```
@Test
void shouldCreateUserSuccessfully() {
    UserRequest newUser = UserRequest.builder()
        .name(faker.name().fullName())
        .job(faker.job().title())
        .build();

    actor.attemptsTo(CreateUser.with(newUser));

    actor.should(
        seeThat(TheStatusCode.was(), equalTo(SC_CREATED)),
        seeThat(TheResponseBody.receivedWas()
            .path("name"), equalTo(newUser.getName()))
    );
}
```

5.3 Modelos y Configuración (20 puntos)

Criterio	Pts	Descripción
Request Models	5	POJOs con @Data, @Builder, @JsonInclude(NON_NULL)
Response Models	5	POJOs con @JsonProperty, @JsonIgnoreProperties(ignoreUnknown = true)
Resources Enum	5	Enum con endpoints como constantes
serenity.conf	5	Base URL configurable por ambiente (default, qa, staging)

5.4 Calidad del Repositorio (20 puntos)

Criterio	Pts	Descripción
README.md	6	Descripción, tech stack, prerequisites, cómo ejecutar, cómo ver reportes
Ejecución limpia	6	./gradlew clean test debe pasar sin errores
Reporte Serenity	4	./gradlew reports debe generar reporte HTML navegable
Code style	4	Código consistente, nombres descriptivos, sin código muerto

6. Bonus (Puntos Extra)

Los siguientes elementos suman puntos adicionales sobre los 100 base:

Bonus	Pts	Descripción
Datafaker	+3	Usar net.datafaker:datafaker para generar datos dinámicos en requests
Spotless	+3	Configurar com.diffplug.spotless con Google Java Format
Manejo de sesión	+2	Usar actor.remember() / actor.recall() para compartir datos entre pasos

7. Entregables

1. URL del repositorio público en GitHub
2. README.md completo con instrucciones de ejecución
3. Screenshot o link del reporte Serenity generado
4. Evidencia de ejecución exitosa (./gradlew clean test pasando)

8. Plazo

10 días hábiles desde la fecha de asignación del reto.

9. Recursos de Referencia

- **Serenity BDD Screenplay:** https://serenity-bdd.github.io/docs/screenplay/screenplay_fundamentals
- **Serenity REST:** https://serenity-bdd.github.io/docs/screenplay/screenplay_rest
- **Reqres API Docs:** <https://reqres.in>
- **JSONPlaceholder:** <https://jsonplaceholder.typicode.com>
- **GoRest:** <https://gorest.co.in>

Nota: Esta propuesta está basada en los patrones y convenciones del proyecto tyba-auto-backend: separación en interactions/api, tasks, questions, models/request|response, utils/constants, uso de RestInteraction, factory methods con instrumented(), EnvironmentConfig con serenity.conf y assertions con seeThat. El objetivo es que el equipo aprenda la arquitectura real que encontrará al incorporarse.